SOFTWARE TEST PLAN

Hospital Management System (HMS)

Comprehensive Master Test Plan Following IEEE Std 829-2008 & Course Template

---

Document Identifier: STP-HMS-2026-V2.0

Document Version: 2.0 (Final Test Plan)

Date: November 5, 2026

Document Status: Final Submission Draft / Pending Approval

Course: Software Engineering Mini-Project

Department: Department of Computer Science and Engineering

Institution: PES University, Bengaluru

Project Team (Authors):

- **Hrushikesh Karnam** (SRN: PES1UG24CS668) — QA Lead

- **Muruli H** (SRN: PES1UG24CS676) — Test Engineer

- **Mandagandla Jhansi** (SRN: PES1UG24CS672) — Developer Lead

- **Neeta R** (SRN: PES1UG24CS680) — Support Lead

Executive Abstract

The Hospital Management System (HMS) is a standalone, single-user C++17 console application designed for clinical administrative staff to manage patient demographic records, physician rosters, and outpatient appointment schedules backed by resilient, local Comma-Separated Values (CSV) persistence. This Software Test Plan (STP) defines the master quality engineering framework, test strategies, environment specifications, verification criteria, and execution roadmap for HMS. It establishes complete bidirectional traceability across all 54 baseline requirements defined in the approved Software Requirements Specification (SRS v1.0), catalogs supplementary sprint implementations, dedicates a specialized subsection to air-gapped security validation, and provides ten representative detailed test case specifications spanning functional, non-functional, and security domains.

Document Control & Revision History

Version	Date	Author(s)	Description of Changes
1.0	20-Oct-2026	QA Team	Initial Software Test Plan baseline defining testing scope, 54 requirements, and 10 representative test cases. *(Note: Cover page printed as 20 November 2026; internal revision history cites 20-Oct-2026).*
2.0	05-Nov-2026	QA Team & Requirements Analysts	Comprehensive final update following the course evaluation test plan template. Incorporated Sprint 1 (31 PRs) and Sprint 2 (31 PRs) enhancements plus post-sprint follow-up PR #127 (63 total merged PRs), updated test environment audit (Windows 11 GCC 15.1.0 clean build and 51/51 CTest verification; Ubuntu 22.04 marked NOT EXECUTED locally), added dedicated Section 5.1 Security Validation (8 baseline + 2 supplementary), complete 54-requirement baseline traceability + 6 supplementary sprint features, and 10 representative detailed test specifications. Status: Final Submission Draft / Pending Approval.

1. Introduction

1.1 Purpose

This document defines the master software test plan for the Hospital Management System (HMS) v2.0. The purpose of this plan is to outline the testing objectives, technical scope, verification strategy, resource allocations, hardware/software environment requirements, execution milestones, team responsibilities, entry/exit criteria, and risk mitigation protocols required to validate that HMS conforms strictly to its approved requirements. It establishes a formal, reproducible testing framework supporting automated unit, integration, system, benchmark, and security verification.

1.2 Scope

Testing encompasses the full functional lifecycle and non-functional quality profile of the Hospital Management System, including:

1. **Patient Management:** Registration, demographic accept-list validation, case-insensitive duplicate ID rejection, search, detailed formatted display, interactive updates with immediate retry, and primary key immutability.

2. **Doctor Management:** Physician onboarding, credential validation, specialty/department management, availability tracking, search, and record updates.

3. **Appointment Management:** Sequential appointment ID generation (`A0001+`), foreign key validation, calendar date/time validation (VR-10, VR-11), double-booking conflict prevention, informational availability queries, slot re-booking after cancellation, chronological sorting, and cancellation confirmation workflows.

4. **Data Persistence & Integrity:** Flat-file CSV parsing, startup loading across missing/empty/corrupted files, malformed line skipping, duplicate/orphan row suppression, immediate atomic disk flushes, 100% data retention across restart, and write-failure fault tolerance.

5. **Console User Interface:** Menu navigation hierarchy with Return option (0), non-numeric and out-of-range input rejection, transactional state protection on failure, standardized user messaging, and clean process termination.

6. **Non-Functional Quality & Security Safeguards:** 20+ input crash resilience vectors, robust startup permutations, latency benchmarks under 1,000 synthetic records, menu depth limits (<= 3), architectural class isolation, C++17 compilation standards, accept-list validation, CSV/formula injection guards, over-length buffer safety, audit record non-deletion, patient privacy minimization, and air-gapped offline security.

System Operational Boundary: The Hospital Management System is designed, implemented, and evaluated strictly as a standalone, single-user C++ console application that persists data in local flat CSV files (`patients.csv`, `doctors.csv`, `appointments.csv`) on the local host filesystem. The application operates in an air-gapped environment without client-server networking, remote web/mobile interfaces, multi-user concurrency, or external SQL database management systems.

Security Requirements Boundary: Baseline SRS v1.0 specifies exactly eight Security Requirements (`HMS-SR-001` through `HMS-SR-008`). During Sprint 2, two supplementary security extensions (`HMS-SR-009` for error message sanitization and `HMS-SR-010` for restricted data directory access) were implemented as proactive architectural hardening enhancements.

1.3 References

1. *HMS Software Requirements Specification (SRS)* v1.0, Software Engineering Mini-Project, September 2026.

2. *HMS System Architecture Document (SAD)* v1.0, October 2026.

3. *HMS Software Test Plan (STP)* v1.0, Document ID: STP-HMS-2026-V1.0, October 2026.

4. *HMS Updated Test Case Document (TCD)* v2.0, Document ID: TCD-HMS-2026-V2.0, Deliverable Part-3, November 2026.

5. *HMS Updated Requirements Traceability Matrix (RTM)* v2.0, Document ID: RTM-HMS-2026-V2.0, Deliverable Part-3, November 2026.

6. *IEEE Std 829-2008*: IEEE Standard for Software and System Test Documentation.

7. *IEEE Std 830-1998*: IEEE Recommended Practice for Software Requirements Specifications.

8. *Local Windows Build & CTest Verification Log*, `DOCS/PART3_TESTING/logs/windows_build_verification.txt`, November 2026.

1.4 Definitions and Acronyms

- **HMS:** Hospital Management System

- **SRS:** Software Requirements Specification

- **STP:** Software Test Plan

- **RTM:** Requirements Traceability Matrix

- **TCD:** Test Case Document

- **FR:** Functional Requirement (`HMS-F-001` through `HMS-F-036`)

- **NFR:** Non-Functional Requirement (`HMS-NF-001` through `HMS-NF-010`)

- **SR:** Security Requirement (`HMS-SR-001` through `HMS-SR-008` baseline; `HMS-SR-009` to `010` supplementary)

- **UAT:** User Acceptance Testing

- **CSV:** Comma-Separated Values flat-file storage

- **CTest:** CMake Automated Test Driver & Test Runner

- **VR:** Validation Rule (domain-level format validation rules VR-01 to VR-14)

- **BR:** Business Rule (system business rules BR-01 to BR-09)

- **DDE:** Dynamic Data Exchange (formula injection vector in spreadsheet-compatible files)

2. Test Items

The specific architectural items, binaries, modules, and persistent artifacts subject to testing comprise:

1. `hms.exe` **(Primary Application Binary):** The compiled, standalone C++ executable implementing all interactive clinical management menus, controllers, and domain logic.

2. **Console User Interface (CLI Menu Subsystem):** The interactive console input/output interface, encompassing main menu dispatch, submenus, validated input prompts, error banners, and formatted record view tables.

3. **Patient Module (**`Patient.h`**,** `Patient.cpp`**):** Domain class managing patient demographic data (ID, name, age, gender, contact, address, medical note), format validation routines (VR-01 to VR-07), and CSV serialization.

4. **Doctor Module (**`Doctor.h`**,** `Doctor.cpp`**):** Domain class managing physician attributes (ID, name, specialization, department, contact, availability), validation routines (VR-08, VR-09), and CSV serialization.

5. **Appointment Module (**`Appointment.h`**,** `Appointment.cpp`**):** Domain class managing appointment metadata (Appointment ID, Patient ID, Doctor ID, Date, Time, Status), calendar validation routines (VR-10, VR-11), and status transitions (`Booked`, `Cancelled`).

6. **HospitalSystem Controller (**`HospitalSystem.h`**,** `HospitalSystem.cpp`**):** Core application controller coordinating menu loops, clinical workflows, entity existence lookups, double-booking conflict detection, chronological sorting, and transactional state protection.

7. **FileManager Subsystem (**`FileManager.h`**,** `FileManager.cpp`**):** Dedicated persistence engine strictly isolating all file stream operations (`fstream`), CSV record parsing, malformed line skipping, duplicate/orphan row suppression, error message sanitization, and atomic non-destructive disk writes.

8. **CSV Persistence Storage:** The local flat-file storage datasets:

- `patients.csv` (Patient demographics and medical notes)

- `doctors.csv` (Doctor credentials, specialties, and schedules)

- `appointments.csv` (Appointment bookings and cancellation audit trail)

9. **Automated Test Executables & Test Harnesses:** The complete automated test suite comprising 49 compiled C++ test executables and 2 custom CMake test drivers (`test_exit_process.cmake`, `test_standalone_offline.cmake`) registered under CTest.

3. Features to be Tested

3.1 Baseline Requirements Mapped to SRS

Table 1 catalogs all 54 baseline requirements defined in the approved HMS Software Requirements Specification (SRS v1.0 Table 7) and establishes their requirement category and testing scope.

Table 1. Features to be Tested Mapped to HMS SRS Requirements

Feature Area	Features to be Tested	SRS Requirement IDs	Requirement Type
Patient Management	Patient Registration: Register patient with auto-ID or valid ID, demographics, contact, address, note	HMS-F-001	Functional
Patient Management	Duplicate Patient ID Rejection: Reject duplicate patient ID case-insensitively and keep records unmodified	HMS-F-002	Functional
Patient Management	Patient Validation Rules: Validate age (0-120), contact (10 digits), non-empty fields, reject comma/newline/formula	HMS-F-003	Boundary / Validation
Patient Management	Patient Search: Search patient by ID or name case-insensitively and display matching records	HMS-F-004	Functional
Patient Management	Patient View & Listing: View patient details by ID and list all registered patients in tabular format	HMS-F-005	Functional
Patient Management	Patient Update: Update patient phone number, address, and medical note with immediate validation and retry	HMS-F-006	Functional
Patient Management	Patient ID Immutability: Prevent editing of Patient ID after registration to preserve referential integrity	HMS-F-007	Functional / Security
Doctor Management	Doctor Registration: Add doctor with ID, name, specialization, department, contact, and initial availability	HMS-F-008	Functional
Doctor Management	Duplicate Doctor ID Rejection: Reject duplicate doctor ID case-insensitively and keep records unmodified	HMS-F-009	Functional
Doctor Management	Doctor Validation Rules: Validate department, specialization, contact (10 digits), and non-empty name	HMS-F-010	Boundary / Validation
Doctor Management	Doctor Search: Search doctor by ID, name, or specialization case-insensitively	HMS-F-011	Functional
Doctor Management	Doctor View & Listing: View doctor details by ID and list all doctors in tabular format	HMS-F-012	Functional
Doctor Management	Doctor Update: Update doctor specialization, department, contact, and availability status	HMS-F-013	Functional
Doctor Management	Doctor ID Immutability: Prevent editing of Doctor ID after registration to preserve referential integrity	HMS-F-014	Functional / Security
Appointment Management	Appointment Booking: Book appointment with auto-generated sequential ID (A0001+) and status Booked	HMS-F-015	Functional
Appointment Management	Patient Foreign Key Validation: Reject appointment booking if Patient ID does not exist in records	HMS-F-016	Functional / Integrity
Appointment Management	Doctor Foreign Key Validation: Reject appointment booking if Doctor ID does not exist in records	HMS-F-017	Functional / Integrity
Appointment Management	Date and Time Validation: Validate calendar date (YYYY-MM-DD, leap years) and time (HH:MM 24-hr clock)	HMS-F-018	Boundary / Validation
Appointment Management	Double-Booking Prevention: Reject booking if doctor already has an active appointment at same date/time	HMS-F-019	Business Logic
Appointment Management	Doctor Availability Check: Query doctor availability for a given date/time without modifying records	HMS-F-020	Functional
Appointment Management	Cancelled Conflict Exclusion: Exclude Cancelled appointments when assessing doctor availability (slot reuse)	HMS-F-021	Functional
Appointment Management	Appointment View & Filter: View appointments filtered by Patient, Doctor, or Date with existence checks	HMS-F-022	Functional
Appointment Management	Appointment Cancellation: Cancel appointment by ID, update status to Cancelled, retain record on disk	HMS-F-023	Functional
Appointment Management	Appointment Chronological Sorting: Sort appointment records chronologically by date and time in views	HMS-F-024	Functional
Persistence & Data Integrity	CSV File Persistence: Store patients, doctors, and appointments in separate local CSV files	HMS-F-025	Persistence
Persistence & Data Integrity	Startup Data Loading: Load records from CSV files on startup across missing/empty/populated states	HMS-F-026	Persistence
Persistence & Data Integrity	Malformed CSV Line Handling: Skip malformed CSV lines without crashing, report skipped count, load valid rows	HMS-F-027	Persistence / Robustness
Persistence & Data Integrity	Immediate Disk Flush: Flush modified records immediately to disk on add, update, and cancel operations	HMS-F-028	Persistence / Integrity
Persistence & Data Integrity	Sequential ID Continuity: Auto-increment ID counter based on existing stored IDs across application restarts	HMS-F-029	Persistence / Integrity
Persistence & Data Integrity	Duplicate Record File Suppression: Ignore duplicate ID records when encountered during CSV startup loading	HMS-F-030	Persistence / Integrity

Feature Area	Features to be Tested	SRS Requirement IDs	Requirement Type
Persistence & Data Integrity	Orphan Appointment File Suppression: Skip appointments with non-existent Patient or Doctor IDs during startup	HMS-F-031	Persistence / Integrity
Persistence & Data Integrity	Non-Destructive File Writes: Prevent data loss or file corruption during failed write operations	HMS-F-032	Persistence / Integrity
Console User Interface	Main Menu Navigation: Display numbered main menu (Patients, Doctors, Appointments, Exit)	HMS-F-033	User Interface
Console User Interface	Return to Main Menu: Provide Option 0 in all submenus to return to Main Menu without state modification	HMS-F-034	User Interface
Console User Interface	Menu Input Validation: Validate menu choices, reject non-numeric and out-of-range inputs gracefully	HMS-F-035	User Interface
Console User Interface	Clear Process Exit: Exit cleanly with status 0 upon selecting Option 4 from Main Menu	HMS-F-036	User Interface
Non-Functional Quality	Console Input Crash Resilience: Withstand 20+ defined edge-case input crash vectors without terminating	HMS-NF-001	Reliability
Non-Functional Quality	File Startup Resilience: Operate normally across missing, empty, read-only, and partially corrupted CSV files	HMS-NF-002	Reliability
Non-Functional Quality	Transactional Input State Protection: Ensure failed operations leave in-memory vectors and CSV files unaltered	HMS-NF-003	Reliability
Non-Functional Quality	Performance Under Load: Startup loading <= 3.0s, search and booking <= 2.0s under 1,000 synthetic records	HMS-NF-004	Performance
Non-Functional Quality	Menu Depth Usability Limit: All operational workflows reachable within <= 3 menu navigation levels	HMS-NF-005	Usability
Non-Functional Quality	Class Architecture Decoupling: Domain classes, controllers, and file manager maintain strict single responsibilities	HMS-NF-006	Architecture
Non-Functional Quality	Cross-Platform C++17 Compilation: Standard ISO C++17 compliant without platform-specific extensions	HMS-NF-007	Portability
Non-Functional Quality	Appointment Conflict Accuracy: 100% correct evaluation of 4-scenario conflict evaluation matrix	HMS-NF-008	Correctness
Non-Functional Quality	Standalone Air-Gapped Deployment: Standalone single-binary executable with zero networking library dependencies	HMS-NF-009	Deployment
Non-Functional Quality	Standard CSV Compatibility: Persistent files are human-readable, standard-compliant CSV files	HMS-NF-010	Interoperability
Security & Privacy	Accept-List Character Validation: Strict allow-list validation for names, IDs, phones, addresses, and notes	HMS-SR-001	Security
Security & Privacy	CSV Delimiter & Formula Injection Defense: Reject commas, newlines, and spreadsheet formula prefixes (=, +, -, @)	HMS-SR-002	Security
Security & Privacy	Buffer Overrun Safety: Safely consume and truncate inputs exceeding 10,000 characters without crashing	HMS-SR-003	Security
Security & Privacy	Untrusted File Input Validation: Validate all records read from disk against domain rules prior to memory intake	HMS-SR-004	Security
Security & Privacy	Atomic File Write Protection: Write through temporary files and flush buffers to prevent partial file corruption	HMS-SR-005	Security
Security & Privacy	Non-Deletion Audit Trail: Retain cancelled appointments permanently on disk for medical audit compliance	HMS-SR-006	Security
Security & Privacy	Patient Data Privacy Minimization: Omit sensitive demographics (phone, address, note) from appointment views	HMS-SR-007	Security / Privacy
Security & Privacy	Air-Gapped Offline Operation: Self-contained binary without external socket calls, telemetry, or remote dependencies	HMS-SR-008	Security

Note on Traceability: Detailed requirement-to-test-case mappings, associated CTest targets, and source files for all 54 baseline requirements are fully detailed in Section 13.1 (Table 9) and the companion publication `DOCS/PART3_TESTING/02_Updated_Requirements_Traceability_Matrix.pdf`.

3.2 Supplementary Sprint Scope Features

During Sprint 1 and Sprint 2, six supplementary capabilities, non-functional quality hardening mechanisms, and security controls were introduced via approved pull requests. To maintain complete academic baseline integrity, these enhancements are tracked separately in Table 2 and do not alter the frozen 54-requirement SRS denominator.

Table 2. Supplementary Sprint Scope Features (Sprint 1 & Sprint 2 Additions)

Supplementary Scope ID	Feature Description	Originating Sprint	Issue / PR	Associated Test Case	Current CTest Target
HMS-SR-009	Filesystem Error Message Sanitization: Omit raw filesystem paths, exception traces, or raw malformed rows from console errors	Sprint 2	Issue #55 / PR #117	`TC-SEC-09`	`TC-SEC-09_SanitizeErrorMessages`
HMS-SR-010	Restricted Data Directory Access: Restrict all file operations strictly to designated data folder; reject traversal paths (..)	Sprint 2	Issue #56 / PR #118	`TC-SEC-10`	`TC-SEC-10_RestrictDataDirectory`
HMS-F-022-EXT	Appointment Filtering Entity Validation: Verify patient/doctor exists before querying appointments when filtering	Sprint 2	Issue #121 / PR #123	`TC-APT-12`	`TC-APT-12_FilterUnknownEntity`
HMS-F-001/008-EXT	Duplicate Patient & Doctor Identity Validation: Detect duplicate individuals via normalized name, contact, specialization heuristics	Sprint 2	Issue #124 / PR #125	`TC-ID-01`	`TC-ID-01_DuplicateIdentityValidation`
HMS-F-023-EXT	Appointment Cancellation Confirmation Workflow: Display appointment summary before confirmation; validate Y/N response	Sprint 2	Issue #120 / PR #122	`TC-APT-11`	`TC-APT-09 / TC-APT-10`
HMS-F-003/010/018-EXT	Universal Immediate Field Validation & Retry: Validate each field immediately and re-prompt same field on error	Sprint 2 / Post-Sprint	Issue #119 / PR #127	`TC-VAL-01`	`Integrated across test suite`

4. Features Not to be Tested

4.1 System Exclusions and Boundaries

To establish explicit testing boundaries and optimize verification resources, specific functional subsystems and architectural designs are excluded from testing. Table 3 enumerates each excluded feature along with the formal engineering justification.

Table 3. Features Excluded from Testing Scope

Feature / Component	Reason for Exclusion
Multi-User Concurrency & Session Locks	HMS is explicitly architected as a single-user desktop console application (Assumption A-01). File locking across concurrent OS processes is not supported.
Client-Server Networking & Remote APIs	The application operates strictly offline without TCP/IP sockets, REST/SOAP APIs, web endpoints, or mobile interfaces (HMS-SR-008).
External SQL Database Engine Integration	Persistence is intentionally implemented using local flat CSV files. RDBMS engines (PostgreSQL, MySQL, SQLite) are out of scope.
Cryptographic At-Rest CSV File Encryption	Data files are stored in plain, human-readable CSV format for portability and clinical transparency (HMS-NF-010). Filesystem-level BitLocker/LUKS encryption is assumed.
User Authentication, Role-Based Access Control (RBAC), Passwords	Operating system-level physical workstation access controls are assumed. Software-level login, passwords, and user session tokens are not implemented.
Medical Billing, Invoicing, Insurance Claims Processing	Financial accounting, billing workflows, and payment gateway integrations are deferred to future enterprise releases.
Electronic Health Record (EHR) HL7 / FHIR Interoperability	Direct integration with external clinical health networks via HL7 or FHIR standards is excluded from this academic release.
Graphical User Interface (GUI) & Web Front-Ends	The application interface is strictly bounded to the standard ANSI/ASCII command-line console interface.

4.2 Distinguishing Out-of-Scope Exclusions from Testing Gaps

It is vital to distinguish architectural out-of-scope exclusions from testing gaps:

- **Out-of-Scope Exclusions (Table 3):** Represent deliberate architectural non-goals (e.g., Billing, SQL database servers, Web APIs) that are not implemented in the codebase by design.

- **Testing Gaps / Build-Time Verifications:** Represent in-scope requirements where verification was constrained by environmental availability. Specifically, requirement `HMS-NF-007` (Cross-Platform Compilation) is an in-scope requirement verified cleanly on the local Windows 11 host with MinGW-w64 GCC 15.1.0, but its Ubuntu Linux 22.04 LTS execution is formally recorded as **NOT EXECUTED** due to local environment unavailability. This is tracked honestly as an infrastructure gap in Section 6, not an out-of-scope exclusion.

5. Test Approach / Strategy

The testing strategy follows a rigorous multi-tier verification methodology designed to ensure requirement compliance, software robustness, data integrity, and security across the entire development lifecycle.

5.1 Security Validation

Security validation is a dedicated, mandatory testing area specifically engineered for a standalone, air-gapped clinical console application. The security testing framework validates the system against local injection attacks, unauthorized file operations, memory safety vulnerabilities, and patient privacy leaks in accordance with SRS Security Objectives (`HMS-SO-01` to `HMS-SO-03`), baseline Security Requirements (`HMS-SR-001` to `HMS-SR-008`), and supplementary security extensions (`HMS-SR-009` and `HMS-SR-010`).

Table 4 details the validation procedures and expected outcomes across all ten security requirements.

Table 4. Baseline and Supplementary Security Requirements Validation

Security Requirement ID	Security Validation Area	Validation Procedure	Expected Result
HMS-SR-001	Accept-List Input Validation	Submit boundary inputs with valid characters (alphanumeric, standard punctuation) and invalid characters (control codes, non-whitelisted symbols).	Valid inputs are accepted; invalid inputs are rejected immediately with rule-specific error message; memory state remains unaltered.
HMS-SR-002	CSV & Formula Injection Guard	Submit patient names, addresses, and notes containing commas, newlines, carriage returns, or spreadsheet formula triggers (=, +, -, @).	Inputs are rejected immediately with exact error 'Input contains illegal character (comma, newline, or formula prefix =, +, -, @)'; CSV uncorrupted.
HMS-SR-003	Over-Length Console Buffer Safety	Feed synthetic strings of 10,000+ characters into Patient Name, Address, and Note console input prompts.	System consumes input stream safely, rejects input as exceeding maximum field bounds, and prompts again with zero memory corruption or crash.
HMS-SR-004	Untrusted CSV File Validation	Load CSV files populated with malformed rows, out-of-range numerical values, truncated fields, and illegal characters.	Invalid rows are safely skipped; skipped record count is reported; valid records load correctly; system maintains integrity.
HMS-SR-005	Non-Destructive Atomic File Writes	Simulate disk write failure (read-only file attribute) during patient registration and appointment booking.	Application reports file write error, remains fully operational, and existing valid file content remains byte-identical.
HMS-SR-006	Non-Deletion Audit Integrity	Cancel an existing appointment via Option 3 (Cancel Appointment) and inspect disk storage in appointments.csv.	Appointment status is changed to 'Cancelled'; the row is preserved on disk; total record count in appointments.csv remains constant.
HMS-SR-007	Patient Data Privacy Minimization	Execute View Appointments by Doctor and View Appointments by Date from console menu.	Appointment listing displays only Appointment ID, Date, Time, Doctor ID/Name, and Patient ID/Name; Patient phone, address, and medical notes are concealed.
HMS-SR-008	Air-Gapped Offline Operation	Perform static dependency inspection and binary symbol analysis (objdump -p hms.exe) for network library linkages.	Binary contains zero imports for socket or networking libraries (ws2_32.dll, libcurl); runs self-contained offline. Note: Static inspection verified offline readiness; active dynamic network packet capture has not been conducted.
HMS-SR-009	Filesystem Error Message Sanitization (Supplementary)	Trigger file I/O failures and observe console error output.	Console displays sanitized high-level diagnostic message without revealing underlying OS filesystem paths, stack traces, or exception dumps.
HMS-SR-010	Restricted Data Directory Access (Supplementary)	Attempt path traversal sequences (../../etc/data) in file operation routines.	System rejects traversal paths and restricts all reads and writes strictly to the designated execution data directory.

5.2 Test Levels

1. **Unit Testing:**

- Focuses on individual C++ classes and isolated algorithmic methods.

- Validates domain entity validation logic (`Patient::validateAge`, `Doctor::validateSpecialization`, `Appointment::validateTime`).

- Verifies input accept-list sanitization against rules VR-01 through VR-14.

- Evaluates string trimming, date/time parsing, and record serialization in isolation.

2. **Integration Testing:**

- Focuses on communication interfaces between subsystems, specifically the interaction boundary between `HospitalSystem` and `FileManager`.

- Validates that modifying operations correctly flush in-memory entity vectors to local CSV files.

- Asserts foreign key validation (e.g., rejecting appointment bookings referencing non-existent Patient or Doctor records).

- Verifies appointment conflict detection logic and slot reuse algorithms upon appointment cancellation.

3. **System Testing:**

- End-to-end black-box verification of the compiled executable (`hms.exe`).

- Simulates realistic user interactive sessions via scripted console input streams.

- Verifies complete navigation hierarchies across main menus and submenus.

- Asserts transactional state protection: guarantees that invalid user inputs leave in-memory vectors and on-disk CSV files completely unaltered.

- Verifies clean application termination with process exit code 0.

4. **Acceptance Testing (UAT):**

- Validates the complete software package against clinical administrative user scenarios.

- Covers end-to-end workflows: patient intake, physician onboarding, appointment scheduling, schedule inspection, and cancellation confirmation.

- *Status Notice:* Formal clinical UAT is documented as planned and pending final course evaluator review and sign-off.

5.3 Test Types

- **Functional Testing:** Verifies all nominal clinical workflows and administrative features against the SRS.

- **Positive & Negative Testing:** Validates that conforming inputs succeed and out-of-spec inputs fail gracefully.

- **Boundary Value Testing:** Exercises demographic boundaries (age 0 and 120, contact numbers of exactly 10 digits, string length maximums).

- **Regression Testing:** Automated re-execution of the 51-target CTest suite after any bug fix or pull request merge.

- **Performance & Benchmark Testing:** Evaluates startup loading latency (<= 3.0s) and search/booking operation latency (<= 2.0s) under 1,000 synthetic records.

- **Usability Testing:** Evaluates menu depth (<= 3 levels), clear formatting prompts, and unambiguous error messaging stating the violated rule.

- **Security & Privacy Testing:** Validates accept-list input domains, formula injection guards, buffer overrun safety, error sanitization, and data privacy minimization.

- **Reliability & Fault Tolerance Testing:** Asserts survival across 20+ input crash vectors and resilience against missing, empty, or corrupt CSV files.

- **Portability Testing:** Confirms standard-compliant compilation under C++17 across supported toolchains.

- **Persistence & Data Integrity Testing:** Confirms atomic file writes, non-destructive failure handling, and 100% record retention after restart.

5.4 Test Design Techniques

- **Equivalence Partitioning:** Dividing input domains into valid and invalid classes (e.g., Patient Age partitioned into valid [0, 120], negative invalid [< 0], and excessive invalid [> 120]).

- **Boundary Value Analysis:** Testing extreme edges of valid ranges (e.g., Age 0, 1, 119, 120, 121; Contact length 9, 10, 11 digits; Time 00:00, 23:59, 24:00).

- **Decision Table Testing:** Evaluating the 4-scenario appointment conflict matrix across Doctor ID, Date, Time, and Status to guarantee 100% conflict detection accuracy.

- **State Transition Testing:** Validating appointment lifecycle states (`Non-Existent` -> `Booked` -> `Cancelled` -> `Slot Re-booked`).

- **Input Validation Testing:** Rejecting non-whitelisted characters, delimiter commas, and formula injection prefixes (`=`, `+`, `-`, `@`).

- **Error Guessing:** Submitting whitespace-only inputs, lowercase IDs, extreme numbers, format specifiers (`%s%d`), and missing terminal newlines.

- **Requirement-Based Testing:** Deriving test case specifications directly from SRS functional and non-functional requirement statements.

5.5 Entry Criteria

Testing phases may commence only when the following entry conditions are satisfied:

1. SRS v1.0 and System Architecture Document (SAD) are baselined and approved.
2. Master Software Test Plan (STP) and Requirements Traceability Matrix (RTM) are reviewed.
3. C++ source codebase in `src/` and `include/hms/` is complete for the target iteration.
4. Build environment (CMake, Ninja/Make, C++17 compiler) is fully operational.
5. Codebase compiles cleanly with zero fatal compilation or linking errors.
6. Baseline synthetic test datasets and test fixtures are prepared.
7. Test execution scripts and harnesses are verified operational.
8. Test execution working directory is configured with isolated scratch folders.

5.6 Exit Criteria

Testing activities for a release milestone are considered complete only when all planned criteria are satisfied:

1. **100% Execution Rate:** 100% of all planned test cases are executed.

2. **Defect Resolution:** 100% of Critical and High-priority defects are resolved, re-tested, and verified closed.

3. **Pass Rate Threshold:** A minimum overall test case pass rate of **95%** is achieved across all executed suites.

4. **Data Integrity:** All persistent CSV files (`patients.csv`, `doctors.csv`, `appointments.csv`) maintain structural integrity and correct comma-separated formatting without corruption after test runs.

(Note: These represent planned target thresholds for evaluation sign-off, not claims of premature completion).

5.7 Regression Strategy

To prevent regressions when bug fixes or feature extensions are merged:

1. Every merged pull request must be accompanied by automated test assertions.

2. The entire automated CTest suite (51 test targets) is executed after any source modification.

3. Critical regression focus areas include: appointment double-booking conflict detection, slot re-booking after cancellation, immediate field retry loops, appointment filtering entity checks, and atomic file write routines.

5.8 Test Automation Strategy

Automated testing is orchestrated via CMake and CTest:

- **Test Executable Suite:** 49 compiled standalone C++ test executables in `tests/` linked against the HMS domain library.

- **CMake Script Test Drivers:** 2 custom CMake script drivers (`tests/test_exit_process.cmake` for process exit code verification; `tests/test_standalone_offline.cmake` for binary objdump symbol inspection).

- **Execution Isolation:** All automated tests execute in dedicated temporary runtime directories (`build/test_*`); root user data files (`patients.csv`, `doctors.csv`, `appointments.csv`) are strictly protected from modification.

6. Test Environment

6.1 Hardware and Platform Specifications

The application is tested across designated development and target execution environments. Table 5 details the environment configurations.

Table 5. Test Environment Specifications

Category	Specification	Purpose
Target Operating Systems	Microsoft Windows 10/11 (64-bit) & Ubuntu Linux 22.04 LTS	Primary execution targets defined in baseline Software Test Plan.
Verified Local OS Host	Microsoft Windows 11 Home Single Language (Version 10.0.22631.0, 64-bit)	Workstation where clean compilation and full CTest suite were executed.
Linux OS Status	Ubuntu Linux 22.04 LTS (Target Specification)	NOT EXECUTED — Ubuntu 22.04 environment not available locally.
Compiler Toolchain	MinGW-w64 GCC 15.1.0 (g++.exe) / MSVC C++17 compatible	Standard C++17 compilation (CMAKE_CXX_STANDARD 17, CMAKE_CXX_STANDARD_REQUIRED ON).
Compiler Flags	-Wall -DCMAKE_BUILD_TYPE=Release	Explicit build-time flags passed to enable all diagnostic warnings and optimization.
Build System	CMake 4.4.4 & Ninja 1.13.2	Orchestrates multi-target compilation and test executable linking.
Automated Test Runner	CTest (bundled with CMake 4.4.4)	Executes 51 automated test targets with failure reporting.
Storage Subsystem	Local filesystem (NTFS on Windows, ext4 on Linux)	Direct file I/O on flat-file CSV tables (patients.csv, doctors.csv, appointments.csv).
Command Shells	Windows PowerShell 5.1+, Windows cmd.exe, Linux Bash	Interactive console execution and batch test harness execution.
Binary Inspection Tools	GNU Binutils (objdump 2.44)	Static symbol inspection for offline air-gapped security validation.

6.2 Test Environment Accuracy & Execution Audit

To maintain absolute academic and engineering honesty, the exact verification status of all execution environments is formally recorded:

- **Local Windows 11 Development Host (VERIFIED):**

- **Operating System:** Microsoft Windows 11 Home Single Language (Version 10.0.22631.0, 64-bit).

- **Toolchain:** MinGW-w64 GCC 15.1.0 (`g++.exe`), CMake 4.4.4, Ninja 1.13.2.

- **Compiler Standard & Flags:** Standard C++17 enforced via CMake (`set(CMAKE_CXX_STANDARD 17)`); compiler flag `-Wall` explicitly passed in the build command (`-DCMAKE_CXX_FLAGS="-Wall"`).

- **Windows Build Verification Output:** All 340 compilation units and executables built cleanly with zero errors; `-Wall` diagnostic activity verified via unused-function warnings.

- **CTest Execution Result: 51 of 51 automated CTest tests passed in 3.13 seconds (100% pass rate)**.

- **Audit Evidence:** Documented in full in `DOCS/PART3_TESTING/logs/windows_build_verification.txt`.

- **Target Linux Environment (Ubuntu 22.04 LTS) (NOT EXECUTED LOCALLY):**

- The local workstation environment does not have Ubuntu Linux 22.04 LTS available locally.

- Requirement `HMS-NF-007` is formally recorded as **NOT EXECUTED — Ubuntu 22.04 environment not available locally**.

- No claim of local Linux test execution is made.

7. Test Schedule

The testing timeline is aligned with the Software Engineering Mini-Project academic lifecycle across Phase 1 (Planning), Phase 2 (Sprints 1 & 2 Execution), and Phase 3 (Deliverables & Final Submission). Table 6 details the milestones.

Table 6. Test Schedule Milestones

Activity / Milestone	Planned Phase	Target Timeline	Status
Requirements & Architecture Baseline	Phase 1	15-Sep-2026 to 30-Sep-2026	**Completed**
Software Test Plan (STP v1.0) Preparation	Phase 1	01-Oct-2026 to 15-Oct-2026	**Completed**
Sprint 1 Implementation & Unit Testing	Phase 2	19-Oct-2026 to 23-Oct-2026	**Completed**
Sprint 2 Implementation & Integration Testing	Phase 2	26-Oct-2026 to 30-Oct-2026	**Completed**
Post-Sprint Follow-up & Immediate Field Retry	Phase 2	31-Oct-2026	**Completed**
Test Harness Consolidation & CTest Automation	Phase 3	01-Nov-2026 to 03-Nov-2026	**Completed**
Deliverable Part-3 Documentation Finalization	Phase 3	04-Nov-2026 to 05-Nov-2026	**Completed**
Final Test Plan (STP v2.0) Review & Submission	Phase 3	05-Nov-2026	**Completed**
Formal Evaluator Viva & Acceptance Sign-off	Phase 3	Pending Academic Schedule	**Pending Review**

8. Test Deliverables

The quality engineering process produces twelve formal deliverables categorized into master plan outputs and specific Part-3 testing artifacts:

8.1 Master Test Plan Outputs

1. **Software Test Plan (STP v2.0):** This master document, available in Markdown (`Software_Test_Plan_Final.md`), Word (`Software_Test_Plan_Final.docx`), and PDF (`Software_Test_Plan_Final.pdf`).

2. **Requirements Traceability Matrix (RTM v2.0):** Complete bidirectional matrix mapping all 54 baseline requirements and 6 supplementary sprint features (`DOCS/PART3_TESTING/02_Updated_Requirements_Traceability_Matrix.pdf`).

3. **Representative Test Case Specifications:** Ten detailed test cases (Tables 11–20) embedded directly within Section 13.2 of this test plan.

8.2 Separate Part-3 Testing Artifacts

4. **Comprehensive Test Case Document (TCD v2.0):** The exhaustive 60-specification test suite covering all baseline and supplementary requirements (`DOCS/PART3_TESTING/01_Updated_Test_Case_Document.pdf`).

5. **Automated Test Scripts & Test Drivers:** 49 compiled C++ test executables in `tests/` and 2 custom CMake script drivers registered in CTest.

6. **Synthetic Test Datasets:** Nominal, boundary, malformed, and 1,000-record volumetric datasets for stress and latency benchmarking.

7. **Test Execution & Build Verification Logs:** Complete compilation logs and CTest execution records (`DOCS/PART3_TESTING/logs/windows_build_verification.txt`).

8. **Defect Tracking Logs & PR Traceability:** Granular records of all 63 pull requests merged across Sprint 1 (31 PRs), Sprint 2 (31 PRs), and post-sprint follow-up PR #127 (1 PR) (`03_Sprint_1_Implementation_Traceability.md`, `04_Sprint_2_Implementation_Traceability.md`).

9. **Granular Test Case Update Change Log:** Formal classification of all test updates into Categories A through E (`05_Test_Case_Update_Change_Log.md`).

10. **Unit Test Execution Reports:** Low-level test execution summaries generated from individual CTest runs.

11. **System & Security Validation Reports:** High-level validation summaries assessing injection resilience, buffer safety, and data persistence.

12. **Final Test Summary & Sign-off Report:** Final evaluative report supporting course viva examination and sign-off.

9. Roles and Responsibilities

Testing responsibilities are distributed among the four project team members according to the official project charter. Table 7 defines the role assignments.

Table 7. Roles and Responsibilities Matrix

Role	Name	SRN / Title	Primary Responsibilities
QA Lead	Hrushikesh Karnam	PES1UG24CS668	Master test plan authoring, IEEE compliance, security test suite oversight, defect triage, sign-off review.
Developer Lead	Mandagandla Jhansi	PES1UG24CS672	Core domain architecture, persistence engine review, unit test harness design, immediate field retry logic.
Test Engineer	Muruli H	PES1UG24CS676	Automated test development, CTest execution, crash resilience testing, performance benchmarks, regression testing.
Support Lead	Neeta R	PES1UG24CS680	Test data synthesis, boundary test fixtures, usability navigation testing, traceability auditing, documentation.

10. Risks and Mitigation

Quality risks inherent in flat-file C++ console applications were analyzed early in the project lifecycle. Table 8 details the risk assessment and active mitigation strategies.

Table 8. Risk Assessment and Mitigation Strategies

Risk Description	Severity	Mitigation Strategy
CSV Data Corruption on System Crash / Power Cut	**High**	Implemented atomic write routines: write to temporary buffer/file first, flush completely, and overwrite existing file only upon successful write completion.
Spreadsheet Formula Injection via Untrusted Input	**High**	Enforced strict sanitization rule VR-14: reject any input string starting with formula execution prefixes (=, +, -, @) or containing embedded commas/newlines.
Appointment Double-Booking Concurrency Conflicts	**High**	Enforced centralized conflict checking in HospitalSystem before any appointment record creation; 100% decision matrix verified across all booking paths.
Unbounded Console Input Stream Buffer Overflow	**Medium**	Implemented safe stream consumption using std::getline with maximum field length bounds (VR-02, VR-12) to reject inputs over 10,000 characters safely.
Orphan Foreign Keys After Application Restart	**Medium**	Enforced strict referential integrity validation on startup loading in FileManager: automatically skip appointment records referencing missing patient/doctor IDs.
Cross-Platform Line Ending Discrepancies (CRLF vs LF)	**Medium**	Standardized CSV line splitting to handle both Windows (\r\n) and Linux (\n) line endings transparently during file parsing.

11. Assumptions and Dependencies

11.1 System Assumptions

1. **Single-User Execution Model:** The Hospital Management System operates strictly as a single-user desktop console application used by one clinic receptionist or administrator at a time (Assumption A-01).

2. **Local Flat-File Storage:** All persistent data is stored in standard ASCII/UTF-8 Comma-Separated Values files (`patients.csv`, `doctors.csv`, `appointments.csv`) on the local host filesystem.

3. **Filesystem Access Rights:** The operating system user executing `hms.exe` possesses standard read and write permissions in the application working directory.

4. **Appointment Availability Logic:** Double-booking conflict detection considers only active appointments (`Booked`); appointments with status `Cancelled` release their associated doctor time slot for immediate re-booking.

5. **Synthetic Test Data:** Performance and boundary testing utilize synthetically generated data that mimics hospital demographics without containing real Protected Health Information (PHI).

11.2 System Dependencies

1. **ISO C++17 Standard Compiler:** Requires a C++17 compliant compiler (MinGW-w64 GCC 9+, MSVC 19.44+, or Linux GCC 11+).

2. **CMake Build System:** Requires CMake 3.20 or newer and CTest for build configuration and automated test target orchestration.

3. **Version Control Integrity:** Depends on Git version control on branch `deliverable/part-3-test-case-rtm-update` rooted at base commit `542b90be8adaddf13f7ce900340973eb4c9f5a56`.

4. **Stable Baseline Specifications:** Depends on approved baseline documents: SRS v1.0, SAD v1.0, and STP v1.0.

12. Suspension and Resumption Criteria

12.1 Suspension Criteria

Formal testing execution shall be immediately suspended under any of the following critical blocking conditions:

1. **Fatal Build / Compilation Failure:** Source code fails to compile or link, blocking generation of `hms.exe` or test binaries.

2. **Immediate Startup Crash:** Application terminates abnormally with an unhandled exception or segmentation fault upon startup across standard file states.

3. **High Defect Severity Blockage:** A critical bug in a core subsystem (`FileManager` or `HospitalSystem`) blocks execution of more than **30%** of remaining planned test cases.

4. **Unrecoverable Test Data Corruption:** Unintended corruption or deletion of root baseline data files that cannot be restored via synthetic seed scripts.

12.2 Resumption Criteria

Testing shall resume only when all resumption prerequisites are verified:

1. The defect causing the build failure, startup crash, or blocking condition is resolved and verified by developer code review.

2. A clean compile and link pass is achieved across all 340 targets.

3. The corrected build passes a standard smoke test (successful startup, main menu display, clean exit).

4. Corrupted test datasets are restored to a known clean state in isolated scratch directories.

13. Test Case Management and Traceability

13.1 Requirements Traceability Matrix (RTM) Baseline

Table 9 establishes complete bidirectional traceability between all 54 baseline SRS requirements, official STP test case IDs, automated CTest targets, test categories, and originating pull requests.

Table 9. Full Baseline Requirements Traceability Matrix (54 Requirements)

Req ID	Requirement Description	STP Test Case ID	Current CTest Target	Test Category	Originating Sprint	Merged PR / Issue
HMS-F-001	Register Patient	TC-PAT-01	TC-PAT-05_PatientDetailsAndRegistrationPersistence	Functional	Sprint 1	PR #61, #80
HMS-F-002	Reject Duplicate Patient ID	TC-PAT-02	TC-PAT-02_DuplicatePatientId	Functional	Sprint 1	PR #62
HMS-F-003	Validate Patient Fields (VR-01 to VR-07, VR-14)	TC-PAT-03	TC-PAT-03_ValidatePatientFields	Boundary	Sprint 1/2	PR #63, #127
HMS-F-004	Search Patient (ID / Name)	TC-PAT-04	TC-PAT-04_SearchPatients	Functional	Sprint 1	PR #67, #70
HMS-F-005	View Patient Details & List	TC-PAT-05	TC-PAT-05_PatientDetailsAndRegistrationPersistence	Functional	Sprint 1	PR #68, #71
HMS-F-006	Update Patient Details	TC-PAT-06	TC-PAT-06_UpdatePatientDetails	Functional	Sprint 1/2	PR #85, #127
HMS-F-007	Prevent Patient ID Edit	TC-PAT-07	TC-PAT-07_PreventPatientIdEdits	Functional	Sprint 1	PR #86
HMS-F-008	Add Doctor	TC-DOC-01	TC-DOC-01_AddDoctor	Functional	Sprint 1	PR #64
HMS-F-009	Reject Duplicate Doctor ID	TC-DOC-02	TC-DOC-02_RejectDuplicateDoctorId	Functional	Sprint 1	PR #65
HMS-F-010	Validate Doctor Fields (VR-01,02,05,08,09,14)	TC-DOC-03	TC-DOC-03_ValidateDoctorFields	Boundary	Sprint 1/2	PR #66, #127
HMS-F-011	Search Doctor (ID / Name / Specialization)	TC-DOC-04	TC-DOC-04_SearchDoctors	Functional	Sprint 1	PR #72
HMS-F-012	View Doctor Details & List	TC-DOC-05	TC-DOC-05_ViewAndListDoctors	Functional	Sprint 1	PR #73
HMS-F-013	Update Doctor Details	TC-DOC-06	TC-DOC-06_UpdateDoctorDetails	Functional	Sprint 1/2	PR #87, #127
HMS-F-014	Prevent Doctor ID Edit	TC-DOC-07	TC-DOC-07_PreventDoctorIdEdits	Functional	Sprint 1	PR #88
HMS-F-015	Book Appointment	TC-APT-01	TC-APT-01_BookAppointment	Functional	Sprint 1	PR #69
HMS-F-016	Reject Invalid Patient FK	TC-APT-02	TC-APT-02_RejectUnknownPatient	Functional	Sprint 1	PR #74
HMS-F-017	Reject Invalid Doctor FK	TC-APT-03	TC-APT-03_RejectUnknownDoctor	Functional	Sprint 1	PR #75
HMS-F-018	Validate Date & Time Format (VR-10, VR-11)	TC-APT-04	TC-APT-04_ValidateAppointmentDateTime	Boundary	Sprint 1/2	PR #76, #127
HMS-F-019	Doctor Double-Booking Conflict Rejection	TC-APT-05	TC-APT-05_TC-NFR-08_PreventDoubleBooking	Logic	Sprint 2	PR #91
HMS-F-020	Check Doctor Availability	TC-APT-06	TC-APT-06_CheckDoctorAvailability	Functional	Sprint 2	PR #92
HMS-F-021	Exclude Cancelled from Conflict Detection (Slot Reuse)	TC-APT-05 (Supp: TC-APT-07)	TC-APT-07_IgnoreCancelledAppointmentsInConflicts	Logic	Sprint 2	PR #93
HMS-F-022	View & Filter Appointments	TC-APT-08	TC-APT-08_ViewAppointmentsFilter	Functional	Sprint 1/2	PR #77, #123
HMS-F-023	Cancel Appointment	TC-APT-09	TC-APT-09_CancelAppointment	Functional	Sprint 2	PR #94, #122
HMS-F-024	Sort Appointments Chronologically	TC-APT-10	TC-APT-10_SortAppointmentsChronologically	Functional	Sprint 2	PR #95
HMS-F-025	Store Entities in Separate CSV Files	TC-PER-01	TC-PER-01_CSVFileCreation	Persistence	Sprint 1	PR #61, #64, #69
HMS-F-026	Load Stored Records on Startup	TC-PER-02	TC-PER-02_MissingCSVFiles	Persistence	Sprint 1	PR #78
HMS-F-027	Skip Malformed CSV Rows on Load	TC-PER-03	TC-PER-03_SkipMalformedCSV	Robustness	Sprint 1	PR #79
HMS-F-028	Flush to CSV on Add/Update/Cancel	TC-PER-04	TC-PER-04_DataPersistenceAcrossRestarts	Persistence	Sprint 1/2	PR #80, #89

Req ID	Requirement Description	STP Test Case ID	Current CTest Target	Test Category	Originating Sprint	Merged PR / Issue
HMS-F-029	Sequential Appointment ID After Restart	TC-PER-05	TC-PER-05_SequentialAppointmentIdRecovery	Persistence	Sprint 1	PR #81
HMS-F-030	Ignore Duplicate ID on Startup Load	TC-PER-06	TC-PER-06_DuplicateIdHandlingInCSV	Integrity	Sprint 1	PR #82
HMS-F-031	Skip Orphan Appointments on Startup Load	TC-PER-07	TC-PER-07_OrphanAppointmentHandling	Integrity	Sprint 1	PR #83
HMS-F-032	Non-Destructive Write on Failure	TC-PER-08	TC-PER-07_HandleCSVWriteFailures	Integrity	Sprint 1	PR #89
HMS-F-033	Main Menu Navigation	TC-UI-01	TC-UI-01_MenuNavigationHierarchy	UI	Sprint 1	PR #84
HMS-F-034	Return to Main Menu (Option 0)	TC-UI-02	TC-UI-02_ReturnToMainMenuOption0	UI	Sprint 1	PR #84
HMS-F-035	Reject Invalid Menu Input	TC-UI-03	TC-UI-03_InvalidMenuChoiceHandling	UI	Sprint 1	PR #84
HMS-F-036	Exit Process Cleanly (Option 4)	TC-UI-04	TC-UI-04_CleanProcessExit	UI	Sprint 1	PR #84
HMS-NF-001	20+ Console Input Crash Resilience	TC-NFR-01	TC-NFR-01_InvalidInputCrashResilience	Reliability	Sprint 1	PR #90
HMS-NF-002	File Startup Resilience (Missing/Corrupt)	TC-NFR-02	TC-NFR-02_MissingEmptyCorruptFiles	Reliability	Sprint 1	PR #78, #79
HMS-NF-003	Transactional In-Memory State Protection	TC-NFR-03	TC-NFR-03_TransactionalStateIntegrity	Integrity	Sprint 2	PR #96
HMS-NF-004	Performance Under 1,000 Records	TC-NFR-04	TC-NFR-04_Performance1000Records	Performance	Sprint 2	PR #125
HMS-NF-005	Menu Depth <= 3 Usability Limit	TC-NFR-05	TC-NFR-05_MenuDepthNavigationLimit	Usability	Sprint 2	PR #102
HMS-NF-006	Class Decoupling (Single Responsibility)	TC-NFR-06	Static Architectural Review & Header Isolation	Architecture	Sprint 2	PR #104
HMS-NF-007	Cross-Platform C++17 Compilation	TC-NFR-07	CMake build-time configuration flags	Portability	Sprint 2	PR #107
HMS-NF-008	100% Conflict Matrix Correctness	TC-NFR-08	TC-NFR-08_AppointmentConflictCorrectness	Correctness	Sprint 2	PR #103
HMS-NF-009	Standalone Deployment (Zero Sockets)	TC-NFR-09	TC-NFR-09_StandaloneOffline	Deployment	Sprint 2	PR #105, #116
HMS-NF-010	CSV Interoperability	TC-NFR-10	TC-NFR-10_HumanReadableCSV	Interoperability	Sprint 2	PR #108
HMS-SR-001	Accept-List Input Validation	TC-SEC-01	TC-SEC-01_AcceptListInputValidation	Security	Sprint 2	PR #109
HMS-SR-002	CSV & Formula Injection Guard	TC-SEC-02	TC-SEC-02_CSVFormulaInjectionGuard	Security	Sprint 2	PR #110
HMS-SR-003	Over-Length Buffer Safety (10k chars)	TC-SEC-03	TC-SEC-03_OverLengthInputSafety	Security	Sprint 2	PR #111
HMS-SR-004	Untrusted CSV File Validation	TC-SEC-04	TC-PER-03_SkipMalformedCSV	Security	Sprint 1/2	PR #79, #112
HMS-SR-005	Non-Destructive File Writes	TC-PER-07	TC-PER-07_HandleCSVWriteFailures	Security	Sprint 1/2	PR #89, #113
HMS-SR-006	Non-Deletion Audit Integrity	TC-APT-09	TC-SEC-06_RetainCancelledAppointmentRecords	Security	Sprint 2	PR #114
HMS-SR-007	Patient Data Privacy Minimization	TC-SEC-07	TC-SEC-07_MinimizePatientDataInAppointmentViews	Security	Sprint 2	PR #115
HMS-SR-008	Air-Gapped Offline Operation	TC-NFR-09	TC-NFR-09_StandaloneOffline	Security	Sprint 2	PR #105, #116

Note on HMS-F-021 Traceability: The baseline STP v1.0 mapped requirement `HMS-F-021` to `TC-APT-05` (Double Booking Prevention and Slot Re-booking). In Sprint 2, PR #93 implemented dedicated automated CTest target `TC-APT-07_IgnoreCancelledAppointmentsInConflicts` specifically focusing on slot re-booking after cancellation. Both the baseline STP ID and the automated supporting target are documented to maintain complete historical and implementation traceability.

Table 10 details the requirements traceability for all six supplementary sprint scope enhancements.

Table 10. Supplementary Sprint Scope Traceability Matrix

Supplementary ID	Feature Description	Originating Sprint	Issue / PR	Associated Test Case	Current CTest Target
HMS-SR-009	Filesystem Error Message Sanitization: Omit raw filesystem paths, exception traces, or raw malformed rows from console errors	Sprint 2	Issue #55 / PR #117	`TC-SEC-09`	`TC-SEC-09_SanitizeErrorMessages`
HMS-SR-010	Restricted Data Directory Access: Restrict all file operations strictly to designated data folder; reject traversal paths (..)	Sprint 2	Issue #56 / PR #118	`TC-SEC-10`	`TC-SEC-10_RestrictDataDirectory`
HMS-F-022-EXT	Appointment Filtering Entity Validation: Verify patient/doctor exists before querying appointments when filtering	Sprint 2	Issue #121 / PR #123	`TC-APT-12`	`TC-APT-12_FilterUnknownEntity`
HMS-F-001/008-EXT	Duplicate Patient & Doctor Identity Validation: Detect duplicate individuals via normalized name, contact, specialization heuristics	Sprint 2	Issue #124 / PR #125	`TC-ID-01`	`TC-ID-01_DuplicateIdentityValidation`
HMS-F-023-EXT	Appointment Cancellation Confirmation Workflow: Display appointment summary before confirmation; validate Y/N response	Sprint 2	Issue #120 / PR #122	`TC-APT-11`	`TC-APT-09 / TC-APT-10`
HMS-F-003/010/018-EXT	Universal Immediate Field Validation & Retry: Validate each field immediately and re-prompt same field on error	Sprint 2 / Post-Sprint	Issue #119 / PR #127	`TC-VAL-01`	`Integrated across test suite`

Companion Documentation Notice: The complete, exhaustive Requirements Traceability Matrix—including source code file paths, test levels, detailed expected results, coverage design status, and execution notes—is published in `DOCS/PART3_TESTING/02_Updated_Requirements_Traceability_Matrix.pdf` and its editable Markdown source `02_Updated_Requirements_Traceability_Matrix.md`.

13.2 Representative Detailed Test Case Specifications

To fulfill the mandatory evaluation template requirement, Table 11 through Table 20 present **exactly ten representative detailed test case specifications**. These specifications cover Functional Requirements (Patient, Doctor, Appointment, Persistence, UI), Non-Functional Requirements (Crash Resilience, Latency Benchmarks, Missing File Resilience), and Security Safeguards (Formula Injection, Privacy Minimization).

(Note: These 10 specifications represent the comprehensive 60-specification test suite documented in `DOCS/PART3_TESTING/01_Updated_Test_Case_Document.pdf`).

Table 11. Test Case Specification: TC-PAT-01 — Patient Registration and Persistence

Attribute	Specification Detail
Test Case ID	`TC-PAT-01`
Requirement ID	`HMS-F-001`
Title	Patient Registration and Persistence
Test Objective	Verify patient registration with valid demographics and immediate persistent writing to patients.csv.
Test Type	Positive Functional
Test Level	System / Integration
Preconditions	hms.exe is running at Main Menu; Patient ID P001 is not present in patients.csv.
Test Data	`ID: P001, Name: John Doe, Age: 45, Gender: M, Contact: 9876543210, Address: 123 Main St, Note: Hypertension`
Execution Steps	1. Select Option 1 (Patient Management) -> Option 1 (Register Patient). 2. Enter Patient ID 'P001'. 3. Enter Name 'John Doe', Age '45', Gender 'M'. 4. Enter Contact '9876543210', Address '123 Main St', Medical Note 'Hypertension'. 5. Confirm save operation.
Expected Result	System displays confirmation message 'Patient registered successfully!'; patients.csv immediately contains the new record row with all 7 fields.
Actual Result	Not Executed (Master test plan baseline; historical test verified via CTest target TC-PAT-05_PatientDetailsAndRegistrationPersistence in Sprint 1).
Execution Status	**Not Executed**
Remarks / Evidence	Core baseline registration test case. Traceable to Issue #1, PR #61, #80. Exercises rule BR-01.

Table 12. Test Case Specification: TC-PAT-02 — Duplicate Patient ID Rejection

Attribute	Specification Detail
Test Case ID	`TC-PAT-02`
Requirement ID	`HMS-F-002`
Title	Duplicate Patient ID Rejection
Test Objective	Verify that attempting to register an existing Patient ID is rejected case-insensitively and leaves stored records unmodified.
Test Type	Negative Functional
Test Level	Unit / Integration
Preconditions	Patient P001 is already registered in patients.csv.
Test Data	`ID: p001 (lowercase), Name: Jane Smith, Age: 30, Gender: F, Contact: 9123456789, Address: 456 Park Ave, Note: None`
Execution Steps	1. Select Option 1 (Patient Management) -> Option 1 (Register Patient). 2. Enter existing Patient ID in lowercase 'p001'. 3. Observe validation check.
Expected Result	System displays error 'Patient ID already exists'; operation halts or prompts for re-entry; patients.csv remains unmodified.
Actual Result	Not Executed (Master test plan baseline; historical test verified via CTest target TC-PAT-02_DuplicatePatientId in Sprint 1).
Execution Status	**Not Executed**
Remarks / Evidence	Enforces primary key uniqueness rule BR-01. Case-insensitivity verified via PR #62.

Table 13. Test Case Specification: TC-APT-01 — Appointment Booking and Validation

Attribute	Specification Detail
Test Case ID	`TC-APT-01`
Requirement ID	`HMS-F-015, HMS-F-016, HMS-F-017, HMS-F-018`
Title	Appointment Booking and Validation
Test Objective	Verify appointment booking with auto-generated sequential ID (A0001+) and foreign key existence validation.
Test Type	Positive / Negative Functional
Test Level	System / Integration
Preconditions	Patient P001 and Doctor D001 exist in database; doctor has no conflicting appointments.
Test Data	`Valid: Patient P001, Doctor D001, Date 2026-11-20, Time 10:00. Invalid FK: Patient P999, Doctor D999.`
Execution Steps	1. Attempt booking with Patient P999. Verify error 'Patient not found'. 2. Attempt booking with Doctor D999. Verify error 'Doctor not found'. 3. Submit valid booking: Patient P001, Doctor D001, Date '2026-11-20', Time '10:00'.
Expected Result	Invalid foreign keys display specific rejection messages without record creation. Valid booking generates sequential Appointment ID, sets status to Booked, and flushes to appointments.csv.
Actual Result	Not Executed (Master test plan baseline; historical test verified via CTest target TC-APT-01_BookAppointment in Sprint 1).
Execution Status	**Not Executed**
Remarks / Evidence	Validates business rules BR-03, BR-04, BR-05. Traceable to Issues #11, #12, #13, PR #69, #74, #75.

Table 14. Test Case Specification: TC-APT-05 — Doctor Double-Booking Conflict Rejection & Cancelled Slot Re-booking

Attribute	Specification Detail
Test Case ID	`TC-APT-05`
Requirement ID	`HMS-F-019, HMS-F-021`
Title	Doctor Double-Booking Conflict Rejection & Cancelled Slot Re-booking
Test Objective	Verify 100% accuracy of double-booking prevention and allow slot re-booking after cancellation.
Test Type	Functional / Business Logic
Test Level	Integration
Preconditions	Doctor D001 has booked appointment A0001 on 2026-11-25 at 14:00.
Test Data	`Patient P002, Doctor D001, Date 2026-11-25, Time 14:00`
Execution Steps	1. Attempt booking Doctor D001 on 2026-11-25 at 14:00 for Patient P002. Observe rejection. 2. Cancel appointment A0001. 3. Attempt booking Doctor D001 on 2026-11-25 at 14:00 for Patient P002 again.
Expected Result	Step 1 displays error 'Doctor not available at specified date and time'; no appointment created. Step 3 succeeds, assigning new ID and status Booked.
Actual Result	Not Executed (Master test plan baseline; historical test verified via CTest targets TC-APT-05_TC-NFR-08_PreventDoubleBooking and TC-APT-07_IgnoreCancelledAppointmentsInConflicts in Sprint 2).
Execution Status	**Not Executed**
Remarks / Evidence	Enforces rules BR-05 and BR-06. Mapped to baseline requirements HMS-F-019 and HMS-F-021. Historical baseline verified via TC-APT-05; supporting slot reuse automated target TC-APT-07 implemented in PR #93.

Table 15. Test Case Specification: TC-APT-04 — Appointment Date and Time Calendar Validation

Attribute	Specification Detail
Test Case ID	`TC-APT-04`
Requirement ID	`HMS-F-018`
Title	Appointment Date and Time Calendar Validation
Test Objective	Verify strict calendar format validation (YYYY-MM-DD, HH:MM) including leap years and 24-hour clock boundaries.
Test Type	Boundary / Format Validation
Test Level	Unit
Preconditions	Application running at Appointment Booking prompt.
Test Data	`Invalid: '2026-02-29' (non-leap), '2026-04-31' (30-day month), '25:00', '10:60', '9:00'. Valid: '2028-02-29', '09:00'.`
Execution Steps	1. Enter invalid date '2026-02-29'. Observe error and retry prompt. 2. Enter invalid date '2026-04-31'. Observe error and retry prompt. 3. Enter valid date '2028-02-29'. Verify acceptance. 4. Enter invalid time '24:00'. Observe error. 5. Enter valid time '09:00'. Verify acceptance.
Expected Result	Invalid calendar dates and out-of-range times are rejected immediately with specific error message naming violated rule; system re-prompts for same field without aborting valid fields.
Actual Result	Not Executed (Master test plan baseline; historical test verified via CTest target TC-APT-04_ValidateAppointmentDateTime in Sprint 1/2).
Execution Status	**Not Executed**
Remarks / Evidence	Enforces rules VR-10 and VR-11. Enhanced in Sprint 2 via PR #127 with immediate field-level retry.

Table 16. Test Case Specification: TC-PER-02 — Missing CSV File Startup Resilience

Attribute	Specification Detail
Test Case ID	`TC-PER-02`
Requirement ID	`HMS-F-026, HMS-NF-002`
Title	Missing CSV File Startup Resilience
Test Objective	Verify application handles missing data files gracefully without crashing, treats them as empty, and creates files on next save.
Test Type	Robustness / Fault Tolerance
Test Level	System
Preconditions	Working directory contains zero CSV files (patients.csv, doctors.csv, appointments.csv deleted).
Test Data	`Fresh application startup with missing files; followed by registering Patient P001.`
Execution Steps	1. Launch hms.exe in empty directory. 2. Observe startup output and main menu. 3. Register Patient P001. 4. Inspect disk directory.
Expected Result	System initializes successfully with 0 records, displays main menu without crashing or throwing unhandled exception; creating a patient generates a new, properly formatted patients.csv.
Actual Result	Not Executed (Master test plan baseline; historical test verified via CTest target TC-PER-02_MissingCSVFiles in Sprint 1).
Execution Status	**Not Executed**
Remarks / Evidence	Satisfies graceful degradation requirement BR-09 and HMS-NF-002. Traceable to Issue #16, PR #78.

Table 17. Test Case Specification: TC-SEC-02 — CSV Delimiter and Formula Injection Guard

Attribute	Specification Detail	
Test Case ID	`TC-SEC-02`	
Requirement ID	`HMS-SR-002`	
Title	CSV Delimiter and Formula Injection Guard	
Test Objective	Verify rejection of inputs containing commas, newlines, or formula execution prefixes (=, +, -, @).	
Test Type	Security Validation	
Test Level	System	
Preconditions	Application running at Patient Name input prompt.	
Test Data	`Input 1: 'Doe, John', Input 2: '=cmd	' /C calc'!A0', Input 3: '+12345', Input 4: '@SUM(A1:A5)'`
Execution Steps	1. Attempt registering patient with Name containing comma 'Doe, John'. 2. Attempt registering patient with Name starting with '='. 3. Attempt registering patient with Name starting with '+'.	
Expected Result	System rejects inputs displaying exact message 'Input contains illegal character (comma, newline, or formula prefix =, +, -, @)'; record is not saved; CSV remains uncorrupted.	
Actual Result	Not Executed (Master test plan baseline; historical test verified via CTest target TC-SEC-02_CSVFormulaInjectionGuard in Sprint 2).	
Execution Status	**Not Executed**	
Remarks / Evidence	Prevents Dynamic Data Exchange (DDE) execution and CSV structural corruption. Traceable to Issue #48, PR #110.	

Table 18. Test Case Specification: TC-SEC-07 — Patient Data Privacy Minimization

Attribute	Specification Detail
Test Case ID	`TC-SEC-07`
Requirement ID	`HMS-SR-007`
Title	Patient Data Privacy Minimization
Test Objective	Verify appointment display views omit sensitive patient demographics (phone number, home address, medical notes).
Test Type	Security / Privacy
Test Level	System
Preconditions	Patient P001 registered with phone '9876543210', address '123 Main St', note 'Hypertension'; Appointment A0001 booked.
Test Data	`Appointment View Menu -> Option 1 View All Appointments.`
Execution Steps	1. Navigate to Appointment Management -> View Appointments. 2. Select Option 1 (View All). 3. Inspect displayed columns and text.
Expected Result	Table displays Appointment ID, Date, Time, Doctor ID/Name, and Patient ID/Name. Patient contact number, address, and medical note are strictly absent from output.
Actual Result	Not Executed (Master test plan baseline; historical test verified via CTest target TC-SEC-07_MinimizePatientDataInAppointmentViews in Sprint 2).
Execution Status	**Not Executed**
Remarks / Evidence	Enforces least-privilege information disclosure and patient confidentiality. Traceable to Issue #53, PR #115.

Table 19. Test Case Specification: TC-NFR-04 — Load and Execution Latency Benchmark Under 1,000 Synthetic Records

Attribute	Specification Detail
Test Case ID	`TC-NFR-04`
Requirement ID	`HMS-NF-004`
Title	Load and Execution Latency Benchmark Under 1,000 Synthetic Records
Test Objective	Verify application meets performance benchmarks: startup loading <= 3.0s and search/booking operations <= 2.0s with 1,000 synthetic records each.
Test Type	Performance / Benchmark
Test Level	System Benchmark
Preconditions	patients.csv, doctors.csv, and appointments.csv each contain exactly 1,000 valid synthetic records.
Test Data	`1,000 synthetic records per file; search query 'P0999'; booking request.`
Execution Steps	1. Measure internal execution latency of displayStartupRecords() and CSV parsing under 1,000 synthetic records using std::chrono::high_resolution_clock. 2. Measure search execution latency for searchPatientById('P0999') under 1,000 records. 3. Measure appointment booking operation latency under 1,000 records. 4. Planned end-to-end OS process startup-to-menu wall-clock latency measurement via external process timing harness.
Expected Result	Internal startup loading and parsing completes in <= 3.0 seconds; search and booking operations complete in <= 2.0 seconds.
Actual Result	Not Executed (Master test plan baseline; historical test verified via CTest target TC-NFR-04_Performance1000Records in Sprint 2 via PR #125).
Execution Status	**Not Executed**
Remarks / Evidence	Internal data loading, search, and booking benchmarks under 1,000 synthetic records were verified via CTest target TC-NFR-04_Performance1000Records in Sprint 2 (PR #125). Full end-to-end OS process startup-to-menu measurement is documented as a planned test procedure pending formal release benchmarking.

Table 20. Test Case Specification: TC-NFR-01 — Console Input Crash Resilience

Attribute	Specification Detail
Test Case ID	`TC-NFR-01`
Requirement ID	`HMS-NF-001`
Title	Console Input Crash Resilience
Test Objective	Verify application robustness and zero crashes across 20+ defined edge-case keyboard input vectors.
Test Type	Reliability / Stress Testing
Test Level	System
Preconditions	Application running at main menu and submenus.
Test Data	`20 invalid vectors: empty lines, spaces, non-numeric strings ('abc', '!@#'), extreme numbers (-9999999, 999999999999), format strings ('%s%d'), control chars.`
Execution Steps	1. Sequentially feed all 20 invalid input vectors to main menu and submenus. 2. Observe process stability, error messages, and prompt returns.
Expected Result	Zero crashes (0 fatal errors / segment violations); system displays appropriate error message for every vector and returns safely to prompt.
Actual Result	Not Executed (Master test plan baseline; historical test verified via CTest target TC-NFR-01_InvalidInputCrashResilience in Sprint 1).
Execution Status	**Not Executed**
Remarks / Evidence	Satisfies reliability objective BR-08 and HMS-NF-001. Traceable to Issue #37, PR #90.

14. Test Metrics and Reporting

14.1 Metric Definitions and Formulas

Testing progress, software quality, and requirement coverage are quantitatively evaluated using standardized metrics:

1. **Test Execution Rate (%):**

Execution Rate = (Total Executed Test Cases / Total Planned Test Cases) * 100

2. **Test Pass Rate (%):**

Pass Rate = (Total Passed Test Cases / Total Executed Test Cases) * 100

3. **Requirement Mapping Coverage (%):**

Requirement Coverage = (Total Requirements Mapped to Test Cases / Total Baselined Requirements (54)) * 100

4. **Defect Density:**

Defect Density = Total Identified Defects / Total Source Lines of Code (KLOC)

5. **Defect Severity Distribution:** Categorization of defects into Blocker, High, Medium, and Low severity tiers.

6. **Defect Aging:** Average elapsed time (in days or hours) between defect logging and verified resolution.

7. **Defect Closure Rate (%):**

Defect Closure Rate = (Total Closed Defects / Total Logged Defects) * 100

14.2 Planned Quality Gate Targets

To qualify for final project approval, the software must achieve the following quality gates:

- **Requirement Mapping Coverage:** Exactly **100.0%** (54/54 baseline requirements mapped).

- **Test Case Execution Rate:** Exactly **100.0%** of all planned test cases executed.

- **Overall Pass Rate:** Minimum **95.0%** pass rate across all executed test suites.

- **Defect Resolution: 0** Critical or High-priority defects remaining open at release.

- **Data File Preservation:** Zero corruption across root-level persistent CSV tables.

15. Approvals

The master Software Test Plan has been prepared, verified, and submitted by the project team for academic review.

Table 21. Document Sign-off and Approval

Role	Name	SRN / Affiliation	Signature / Status	Date
QA Lead	Hrushikesh Karnam	PES1UG24CS668	Submitted for Review	05-Nov-2026
Developer Lead	Mandagandla Jhansi	PES1UG24CS672	Submitted for Review	05-Nov-2026
Test Engineer	Muruli H	PES1UG24CS676	Submitted for Review	05-Nov-2026
Support Lead	Neeta R	PES1UG24CS680	Submitted for Review	05-Nov-2026
Course Evaluator	Mini-Project Evaluator	Dept of CSE, PES University	Pending Approval	—

Notice: This document represents the final submission draft prepared by the project engineering team. Formal academic evaluator approval remains pending review.